

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky
Katedra počítačov a informatiky

Zvýšenie bezpečnosti vybraných šifrovacích algoritmov v prístupových systémoch

Diplomová práca

ACCESS-SECURITY

Systémová príručka

Vedúci diplomovej práce:

TODO — doplniť vedúceho práce

Diplomant:

Oleksandr Mykhailyshyn

Konzultant diplomovej práce:

Košice 2026

Contents

1	Funkcia programu	1
2	Analýza riešenia	2
3	Popis programu	2
3.1	Popis riešenia	3
3.2	Popis algoritmov a údajových štruktúr, globálnych premenných . . .	4
3.3	Popis modulov a podprogramov	9
4	Zoznam použitej literatúry	16

1 Funkcia programu

Access-Security je prototyp systému kontroly fyzického prístupu na báze RFID/NFC. Jeho úlohou je autorizovať držiteľov kariet na prechod cez elektronicky riadené dvere na základe kombinácie roly karty a role dverí, pričom celý komunikačný reťazec medzi čítačkou a serverom je chránený kryptograficky — autentifikovaným šifrovaním (AEAD), anti-replay ochranou, HKDF per-reader deriváciou kľúčov, detekciou protokolových anomálií a tamper-evident auditom.

Systém tvoria tri hlavné časti:

- **Hardvérová čítačka** postavená na mikrokontroléri ESP32 a module RC522. Číta UID karty, pripojí identifikátor čítačky a dverí, doplní monotónne počítadlo `hw_seq` a cez bezdrôtové pripojenie Wi-Fi odošle autentifikovanú HTTP požiadavku (HMAC-SHA-256 podpis nad metódou, cestou a telom) na server.
- **Serverová aplikácia** v jazyku C++20, pozostávajúca z dvoch nezávislých HTTP služieb: *frontend* služba overuje HMAC podpis požiadavky a konštruje interný AEAD frame, *DecisionService* frame dešifruje, overí anti-replay, spustí anomálny detektor, vyhodnotí autorizáciu (RBAC) a výsledok zapíše do auditného logu. Lokálne úložisko je realizované cez SQLite v WAL móde.
- **Webové administrátorské rozhranie** pre správu čítačiek, kariet, rolí dverí, kontrolu auditného logu s overením integrity reťaze a sledovanie prevádzkových udalostí v reálnom čase.

Program je určený na nasadenie v prostredí, kde konvergujú požiadavky fyzickej a informačnej bezpečnosti — tam, kde sa od prístupového systému očakáva ochrana nielen proti jednoduchému skopírovaniu karty, ale aj proti aktívnym útokom na komunikačný kanál (replay, manipulácia hlavičky, podsunutie nonce, rollback sekvenčného čísla, forgery pokusy o AEAD tag a pod.).

2 Analýza riešenia

Detailný teoretický rozbor problematiky — analýza aktuálnych prístupov ku kontrole fyzického prístupu, model hrozieb (STRIDE), porovnanie kryptografických primitív (ChaCha20-Poly1305 vs. AES-GCM, stratégie generovania nonce, HKDF derivácia kľúčov, HMAC autentifikácia), zásady návrhu bezpečných protokolov, ochrana súkromia identifikátorov, tamper-evident auditovateľnosť, bezpečnosť vstavaného zariadenia a bezpečnosť transportnej vrstvy — je obsahom kapitoly 2 *Analýza* hlavnej diplomovej práce, ktorá je súčasťou rovnakého repozitára.

Táto systémová príručka sa zameriava na technickú dokumentáciu konkrétnej implementácie: organizáciu kódovej bázy, algoritmy realizované vlastným kódom, údajové štruktúry, konfiguráciu a spôsob zostavenia a spustenia systému. Pre úplné odôvodnenie zvolených kryptografických primitív a architektonických rozhodnutí je čitateľ odkázaný na hlavnú diplomovú prácu.

3 Popis programu

Program pozostáva z troch vrstiev. Hardvérová vrstva `hw_layer` je firmvér bežiaci na ESP32 s čítačkou MFRC522; jeho úlohou je čítanie UID karty a odoslanie autentifikovanej HTTP požiadavky serveru. Serverová časť je písaná v jazyku C++20 a je rozdelená na dve nezávislé HTTP služby: *frontend* (port 8080), ktorá prijíma požiadavky od čítačiek, overuje HMAC podpis a konštruuje interný AEAD frame, a *DecisionService* (port 8081), ktorá frame dešifruje, spúšťa anti-replay kontrolu, anomálny detektor a RBAC vyhodnotenie a zapisuje výsledok do auditného logu. *DecisionService* počúva výhradne na loopback rozhraní `127.0.0.1`, takže je dostupný len lokálnej frontend službe. Webové administrátorské rozhranie slúži pre správu čítačiek, kariet a rolí dverí a pre sledovanie prevádzky v reálnom čase.

Rozhodovacia cesta (`DecisionEngine::handleFrameBytes`) je organizovaná od

najlacnejších kontrol po najdrahšie, aby sa neplatné požiadavky odfiltrovali čo najskôr: parsovanie → validácia čítačky → kontrola verzie kľúča → anti-replay (read-only) → AEAD dešifrovanie → anomálny detektor → RBAC → audit.

3.1 Popis riešenia

Projekt sa skladá z troch komponentov, ktoré dokopy tvoria funkčný prístupový systém spolu s experimentálnou platformou pre jeho vyhodnotenie. Bezpečnostné mechanizmy implementované v serverovej časti (AEAD frame s AAD väzbou, HKDF per-reader derivácia kľúčov, deterministický nonce, anti-replay sliding window, detektor protokolových anomálií s karanténou, tamper-evident audit log a pseudonymizácia identifikátorov kariet) sú podrobne popísané a odôvodnené v kapitole 3 hlavnej diplomovej práce; tu sú spomenuté len v rozsahu potrebnom pre orientáciu v kóde.

Hardvérová vrstva. Firmvér pre ESP32 s čítačkou MFRC522, písaný pre Arduino-ESP32 framework. Číta UID karty, udržiava monotónne počítadlo `hw_seq` v perzistentnom NVS úložisku, a odošle autentifikovanú HTTP požiadavku cez Wi-Fi (HMAC-SHA-256 podpis nad reťazcom `metóda + cesta + telo`). Detailné zapojenie a popis firmvéru je v následných podkapitolách.

Serverová aplikácia. Dve nezávislé HTTP služby v jazyku C++20: *frontend* (port 8080) prijíma hardvérové požiadavky, overuje HMAC podpis a konštruje interný AEAD frame; poskytuje aj administratívne API a servuje webové rozhranie. *DecisionService* (port 8081, viazaný na loopback) frame dešifruje, spúšťa pipeline kontrol (anti-replay, anomálny detektor, RBAC, audit) a vracia rozhodnutie. Úložisko je SQLite v režime WAL.

Webové administrátorské rozhranie. Vanilla JavaScript (bez frameworku) servovaný frontend službou. Spravuje čítačky, karty a role dverí, verifikuje integritu

auditného logu a sleduje prevádzkové udalosti v reálnom čase (polling každých 500 ms; auditný log auto-refresh každú sekundu).

Experimentálna a analytická vrstva. C++ harness (`experiments/`) zdieľa produkčný kód kryptografickej a rozhodovacej vrstvy, čím sa zaručuje, že merania odrážajú skutočné správanie systému. Obsahuje sedem scenárov pokrývajúcich triedy útokov skúmaných v hlavnej práci. Python skripty (`analysis/`) spracúvajú CSV výstupy a generujú grafy a LaTeX-ready tabuľky pomocou `pandas`, `matplotlib` a `seaborn`.

Vetvenie repozitára. Projekt je udržiavaný v dvoch vetvách: `main` (produkčná, obsahuje len funkčný systém) a `analytical` (rozšírená o moduly `experiments/` a `analysis/`). Rozdiel je čisto aditívny — produkčný kód je identický, `analytical` iba dopĺňa nástroje pre meranie, čím je zaručené, že experimenty bežia na presne tom istom kóde, aký by bol nasadený v produkcii.

3.2 Popis algoritmov a údajových štruktúr, globálnych premenných

Táto podkapitola opisuje vlastné algoritmy implementované v projekte, kľúčové údajové štruktúry, rozdelenie kódu na moduly a model interakcie s používateľom.

Modulárna štruktúra. Serverová časť je rozdelená do deviatich modulov so statickými knižnicami a jednosmernými závislosťami: `crypto_lib` (AEAD, HKDF, generátory nonce, HMAC), `protocol_lib` (binárny formát frame, replay window), `key_manager` (derivácia kľúčov, rotácia), `access_core` (FrameHandler, ProtocolAnomalyDetector), `access_decision` (DecisionEngine, RBAC), `access_storage` (SQLite, audit log), `config_loader` (YAML), `runtime_events` (udalosti pre web) a `access_admin` (HTTP server, služby, web frontend). Každý modul má vlastnú sadu unit testov v podadresári `tests/`.

Algoritmus: **rozhodovacia pipeline.** Hlavný algoritmus serverovej časti je obsiahnutý v `DecisionEngine::handleFrameBytes` (a podriadenej `FrameHandler::handle`). Spracovanie frame prebieha v pevne definovanom poradí od najlacnejších kontrol po najdrahšie, aby sa neplatné požiadavky odfiltrovali čo najskôr:

1. **Parsovanie frame** — validuje magic, verziu, hlavičku, ciphertext (s kontrolou maximálnej dĺžky pred alokáciou), tag a absenciu trailing bajtov.
2. **Validácia čítačky a verzie kľúča** — overenie, že `reader_id` je registrovaný a `key_version` je aktuálna alebo (pri povolenej tolerancii) bezprostredne predchádzajúca.
3. **Anti-replay kontrola** — *read-only* volanie `ReplayWindow::contains(seq)`; pri nájdení duplicitu sa frame odmietne bez pokusu o dešifrovanie.
4. **AEAD dešifrovanie** — derivácia per-reader kľúča cez HKDF, overenie Poly1305 tagu nad hlavičkou (AAD) a ciphertextom.
5. **Anomálny detektor** — pri úspešnom dešifrovaní vyhodnotenie štyroch protokolových invariantov (viď nižšie).
6. **Zaznamenanie sekvencie (remember)** — až po úspešnom kroku 4 sa `seq` pridá do replay window, aby neplatné frame nemohli „otraviť“ stav.
7. **RBAC autorizácia** — vyhľadanie karty podľa HMAC odtlačku, overenie väzby čítačka $\leftrightarrow$ dvere a povolenia role pre cieľové dvere.
8. **Zápis do auditného logu** — bez ohľadu na výsledok rozhodnutia (allow alebo deny).

Algoritmus: **sliding replay window.** `ReplayWindow` udržiava dve komplementárne údajové štruktúry — `std::deque<uint64_t> _order` (FIFO

poradie vloženia) a `std::unordered_set<uint64_t> _seen` (rýchle $O(1)$ vyhľadávanie) — plus ukazovateľ `_maxSeen` na najvyššiu videnú hodnotu. Pri každom `remember(seq)`: ak je `seq > maxSeen`, aktualizuje sa ukazovateľ; následne sa `seq` pridá do oboch kontajnerov. Keď veľkosť okna prekročí limit (`evictOldestIfFull`), najstaršia hodnota z `_order` sa vyhodí aj z `_seen`. Kontrola `contains(seq)` vracia `true`, ak je `seq ≤ maxSeen - windowSize` (príliš staré) alebo prítomné v `_seen`. Táto konštrukcia dosahuje amortizovanú $O(1)$ zložitosť pre všetky operácie a zároveň toleruje krátke preusporiadania.

Algoritmus: deterministický nonce (HMAC konštrukcia). Generátor v triede `HmacNonceGenerator` vypočíta HMAC-SHA-256 nad kanonicky serializovaným kontextom hlavičky (`reader_id`, `door_id`, `seq`, `key_version`, `ts_unix_ms`) so samostatným nonce kľúčom odvodeným cez HKDF. Výstup HMAC má 32 bajtov; prvé `nonce_len` bajtov (12 pre ChaCha20-Poly1305, 24 pre XChaCha20-Poly1305) sa skopírujú ako nonce, ostatok sa vynuluje funkciou `sodium_memzero` pred opustením zásobníka. Prijímacia strana prepočíta očakávanú hodnotu rovnakým postupom a porovná ju s prijatou cez `sodium_memcmp` (porovnanie v konštantnom čase, odolné voči timing útokom).

Algoritmus: detektor anomálií ako stavový automat. Detektor udržiava per-reader stav s tromi položkami: najvyššia videná sekvencia (`maxSeenSeq`), počítadlo po sebe idúcich zlyhaní tagu (`consecutiveTagFails`) a stav karantény. Pri reporte sekvencie sa overí, či `seq + rollbackThreshold < maxSeenSeq`; ak áno, čítačka je zakarantenovaná s dôvodom `seq_rollback`, inak sa `maxSeenSeq` aktualizuje. Pri zlyhaní tagu sa inkrementuje počítadlo a po dosiahnutí limitu (`tagFailStreakLimit`) sa aktivuje karanténa `tag_fail_streak`. Replay a nonce mismatch karanténujú okamžite. Úspešné spracovanie vynuluje počítadlo zlyhaní (legitímne frame nesmú „frustrovať“ streak). Karanténa pretrváva až do manuálneho zrušenia cez administrátorské API.

Algoritmus: tamper-evident audit chain. Pri zápise sa každá auditná udalosť serializuje do kanonického byte reťazca (*ts*, *reader_id*, *door_id*, *seq*, *allow*, *reason*, *card_hmac*, *action* — v pevne stanovenom poradí s little-endian kódovaním a prefixmi dĺžky pre reťazce). Hash záznamu je vypočítaný ako $HMAC(K_{audit}, prev_hash || fields)$ a uložený v stĺpci *entry_hash*; predošlý hash je v stĺpci *prev_hash*. Súčasne sa hash nového záznamu uloží aj do tabuľky *audit_anchor* ako ochrana proti truncation. Verifikácia rekonštruje reťazec od prvého záznamu, porovnáva každý vypočítaný hash so *storedEntry/storedPrev* z DB a na konci overí zhodu s anchor hodnotou — akákoľvek modifikácia, vloženie alebo odrezanie záznamov sa odhalí.

Algoritmus: pseudonymizácia identifikátora karty. Komponent *CardIdHasher* vypočíta HMAC-SHA-256 nad hex reťazcom UID karty s pepper kľúčom a vráti výsledok v hex kódovaní. Pepper je per-verziu odvodený z master kľúča cez HKDF, čo umožňuje rotáciu pepperu bez straty continuity. Vyhľadanie karty v databáze prebieha tak, že server spočíta HMAC s aktuálnym pepperom a hľadá v stĺpci *cards.card_hmac*; ak nenájde, opakuje s predchádzajúcou verziou pepperu (pre tolerantný prechod počas rotácie).

Algoritmus: parsovanie binárneho frame. Parser binárneho frame je implementovaný defenzívne: najprv kontroluje minimálnu veľkosť bufferu, potom magic reťazec (AS01), verziu protokolu, následne po poliach parsuje hlavičku (fixné pozície s little-endian kódovaním), nonce, dĺžku ciphertextu (s kontrolou pred alokáciou, čím sa zabráni DoS útoku cez nadmernú dĺžku), samotný ciphertext, 16-bajtový tag a na záver kontroluje absenciu trailing bajtov. Každá neplatná pozícia vyhodí výnimku s presným popisom chyby; ošetrovanie chyby je centralizované v rozhodovacej pipeline.

Konfigurácia. Behová konfigurácia je udržiavaná v YAML súboroch v adresári *config/*. Hlavný produkčný profil *access_security.yaml* obsahuje

piate skupiny parametrov (`frame_handler`, `key_management`, `storage`, `admin`, `experiment`). Okrem neho existuje šesť experimentálnych profilov `profile_a{1,2}_r{0,1,2}.yaml`, ktoré kombinujú dva AEAD algoritmy a tri stratégie generovania nonce.

Modul `config_loader` poskytuje funkciu `loadFromYaml(path) → Config`, ktorá načíta YAML a validuje hodnoty. Výsledná štruktúra `Config` agreguje päť podštruktúr: `FrameHandlerConfig` (anti-replay, max skew, väzby), `StorageConfig` (cesta k SQLite), `AdminConfig` (HTTP port, admin token, HMAC secret, `decision_mode`), `KeyManagementYaml` (verzia kľúča, cesta k master key) a `ExperimentConfig` (AEAD cipher, nonce stratégia, detektor, kotva auditu). Všetky polia majú rozumné predvolené hodnoty, takže konfigurácia je priamočiara a zrozumiteľná bez dokumentácie.

Schéma databázy. Úložisko je realizované cez SQLite v režime WAL. Schéma obsahuje šesť tabuliek (tabuľka 3–1), ktoré sú pri prvom spustení automaticky vytvorené modulom `access_storage` cez `CREATE TABLE IF NOT EXISTS`.

Table 3 – 1 Schéma SQLite databázy

Tabuľka	Obsah
<code>cards</code>	HMAC(pepper, UID) → rola karty
<code>door_roles</code>	Povolené roly pre každé dvere (RBAC)
<code>readers</code>	Registrované čítačky s verziou kľúča
<code>reader_doors</code>	Väzba čítačka ↔ dvere (FK, CASCADE)
<code>audit_log</code>	Záznamy auditu s HMAC hash chain
<code>audit_anchor</code>	Ochranná kotva proti truncation

Žiadna tabuľka neobsahuje surové UID kariet — ukladá sa len HMAC odtlačok s pepperom. Tabuľky `audit_log` a `audit_anchor` spolu tvoria tamper-evident auditný mechanizmus opísaný v podkapitole 3.1.

Globálne premenné. Projekt sa zámerne vyhýba globálnym mutable premenným. Jediný agregujúci objekt serverovej aplikácie je inštancia `AppState` (v module `access_admin`), ktorá drží zdieľaný stav — `DecisionEngine`, `KeyManager`, úložisko, kruhový buffer prevádzkových udalostí — a dva mutexy: `app.m` pre administratívne API a `app.engineMutex` pre rozhodovaciu cestu. Všetky modifikácie zdieľaného stavu sú chránené jedným z týchto mutexov. Firmvérová časť udržiava len statické premenné v rámci hlavného skechu (`hwSeq`, `lastUid`, `lastSendMs`) — plne opísané sú v následnej podkapitole o hardvérovej vrstve.

Dialóg s používateľom. Interakcia s administrátorom prebieha výhradne cez webové rozhranie: používateľ v prehliadači otvorí administrátorskú stránku (servovanú frontend službou na porte 8080), zadá voliteľný administrátorský token a prepína medzi sekciami *Readers*, *Door roles*, *Cards*, *Audit*, *Runtime logs* a *DB Export/Import*. Každá operácia (pridanie karty, zrušenie roly, verifikácia auditu) je realizovaná volaním REST endpointu; výsledok je zobrazený v zodpovedajúcej tabuľke alebo stavovom paneli. Dialóg s používateľom karty (t.j. samotné priloženie karty k čítačke) je realizovaný cez svetelnú a zvukovú signalizáciu opísanú v následnej podkapitole o hardvérovej vrstve.

3.3 Popis modulov a podprogramov

Táto podkapitola sumarizuje verejné rozhranie každého modulu — najdôležitejšie triedy a funkcie, ich vstupno-výstupné parametre a vzťah k ostatným modulom. Globálne premenné jednotlivé moduly nepoužívajú; všetok zdieľaný stav je centralizovaný v objekte `AppState` (popísaný v 3.2). Poradie popisu zodpovedá závislostiam: od základných knižníc po vyššie vrstvy.

hw_layer (firmvér ESP32). Súbor `hw_layer.ino` obsahuje obvyklú dvojicu Arduino funkcií. `setup()` vykoná inicializáciu (sériová konzola, GPIO, LEDC, NVS, SPI, MFRC522, Wi-Fi). `loop()` periodicky volá `rfid.PICC_IsNewCardPresent()`

a `rfid.PICC_ReadCardSerial()`; pri detekcii karty zavolá `postUidToServer(uid, allow)`. Tá overí Wi-Fi, inkrementuje `hw_seq`, zostaví JSON telo, vypočíta HMAC podpis (`hmacSha256Hex`) a odošle požiadavku cez `HTTPClient`. Výsledok je signalizovaný cez `signalAllow()` alebo `signalDeny()` (LED + LEDC tón).

crypto_lib. Trieda `SecureAead` (`secure_aead.hpp`) zapuzdruje `ChaCha20-Poly1305` alebo `XChaCha20-Poly1305` z knižnice `libsodium`. Verejné šifrovacie metódy: `sealWithSeq(plaintext, aad, seq)` (interne odvodí nonce zo `seq`) a `sealWithNonce(plaintext, aad, nonce)` (explicitne dodaný nonce) — obe vracajú `Ciphertext` obsahujúci `ciphertext`, `tag` a `nonce`. Dešifrovanie je metódou `openWithNonce(ct, tag, aad, nonce)`, ktorá vracia `std::vector<uint8_t>` s plaintextom alebo vyhodí výnimku pri zlyhaní tag verifikácie. Trieda `Hkdf` (`hkdf.hpp`) implementuje HKDF-SHA-256 podľa RFC 5869: konštruktor vykoná `extract` fázu (`PRK = HMAC(salt, IKM)`), metóda `derive()` → `vector<uint8_t>` `expand` fázu. Generátory nonce (`nonce_generator.hpp`) implementujú spoločné rozhranie `INonceGenerator::generate(context, seq) → array<uint8_t,24>`; konkrétne triedy `RandomNonceGenerator` (`libsodium randombytes_buf`) a `HmacNonceGenerator` (HMAC nad kontextom).

protocol_lib. `Frame` (`frame.hpp`) je dátová štruktúra obsahujúca `Header`, `ciphertext` (`vector<uint8_t>`) a `Tag16`. Voľná funkcia `serialize(const Frame&) → vector<uint8_t>` vyprodukuje binárny tvar (`magic`, `verzia`, `hlavička`, `dĺžka ciphertext`, `ciphertext`, `tag`); `parseFrame(bytes, maxCtLen=4096) → Frame` robí opačnú operáciu cez triedu `FrameParser` s ochranou proti DoS útoku. Trieda `ReplayWindow` (`replay_window.hpp`) poskytuje `contains(seq) → bool`, `remember(seq)` a `accept(seq) → bool` (atomická kontrola + vloženie).

key_manager. Trieda `KeyManager` (`key_manager.hpp`) odvodzuje podkľúče z master kľúča. Konštruktor prijíma 32-bajtový `MasterKey` a `KeyManagerConfig` (aktuálna verzia, povolenie predošlej verzie). Verejné metódy:

- `deriveAeadKey(readerId, keyVersion)` — HKDF AEAD kľúč so saltom obsahujúcim ID čítačky a verziu,
- `deriveNonceKey(readerId, keyVersion)` — per-reader kľúč pre HMAC nonce konštrukciu,
- `deriveAuditHmacKey()` — per-inštanciu kľúč pre auditný hash chain (bez parametrov),
- `deriveCardPepper(keyVersion)` — pepper pre pseudonymizáciu kariet,
- `isAcceptedKeyVersion(keyVersion)` — kontrola, či je verzia akceptovaná pre dešifrovanie,
- `masterAsAeadKey()` — priamy režim bez HKDF (pre testovanie).

access_core. `FrameHandler::handle(frameBytes) → HandleResult` je centrálna metóda spracovania frame: parsuje, validuje čítačku a verziu kľúča, volá `ReplayWindow::contains`, deriváciu kľúča a AEAD dešifrovanie. `HandleResult` obsahuje pole `allow`, `reason` (string code), `plaintext` a `header`. `ProtocolAnomalyDetector` (`protocol_anomaly_detector.hpp`) ponúka per-reader päť report metód: `reportReplay`, `reportSeq`, `reportNonceMismatch`, `reportTagFailure` a `reportSuccess`; spolu s dotaznými `isQuarantined` a `quarantineInfo`. Trieda je *thread-safe* (interný mutex).

access_decision. Hlavná metóda `handleFrameBytes` (s parametrami `frameBytes` a mapou `replay window` podľa čítačky) vracia `DecisionResult` a orchestruje úplný rozhodovací cyklus: volá `FrameHandler::handle`, vyhodnotí stav anomálneho detektora, prevedie pseudonymizáciu UID cez `CardIdHasher::hmacHex`, vykoná RBAC vyhľadanie cez `IAccessStore`, zapíše do `IAuditLog` a vyšle udalosť do voliteľnej `EventBus`. Štruktúra `DecisionResult` obsahuje `allow`, `reason` (kódy ako `ok`, `replay`, `decrypt_failed`, `unknown_card`,

forbidden, quarantined). `CardIdHasher::hmacHex(uid) → string` vypočíta hex HMAC s pepperom.

access_storage. `SqliteAccessStore` implementuje rozhranie `IAccessStore` s CRUD operáciami nad tabuľkami `readers`, `cards`, `door_roles`, `reader_doors`. `SqliteAuditLog` implementuje `IAuditLog::append(event)`; interne pri zápise volá `getLastEntryHash()` a `computeEntryHash(prev, event)` pre tvorbu HMAC chain a aktualizuje tabuľku `audit_anchor`. Verifikácia (`verifyAuditChain(db, key)` v `audit_verify.cpp`) prečíta všetky záznamy v poradí a rekonštruuje reťazec; vracia `AuditVerifyResult {bool ok, int64_t bad_id, std::string error}` — pri prvej zistenej nezhode sa verifikácia zastaví a vráti ID problémového záznamu spolu s popisom chyby.

Pomocné moduly. Modul `config_loader` obsahuje jedinú verejnú funkciu `loadFromYaml(path)`, ktorá načíta YAML súbor cez `yaml-cpp` a vráti naplnenú štruktúru `Config`. `runtime_events` poskytuje triedu `EventBus` s kruhovým bufferom prevádzkových udalostí (`push(event)`, `getAfter(afterId, limit)`) pre real-time zobrazovanie vo webovom rozhraní. `access_admin` obsahuje vstupný bod `main()` (spustí dva HTTP servery), centrálny `AppState` a service vrstvu (`hw_service`, `cards_service`, `readers_service`, `audit_service` a ďalšie), ktorá tvorí tenkú adaptáciu medzi HTTP požiadavkami a jadrom systému.

Webové rozhranie (web/). Vanilla JavaScript priamo servovaný frontend službou. Modul `app.js` obsluhuje navigáciu medzi sekciami a spúšťa `startEventPolling()` (interval 500 ms) a `startAuditAutoRefresh()` (interval 1 s, aktívny iba pri otvorenej audit sekcii). Jednotlivé súbory `readers.js`, `cards.js`, `doors.js`, `audit.js`, `db.js` implementujú obslužné funkcie pre svoje sekcie; všetky volajú jednotné API helper `api(path, opts)` z `utils.js`, ktoré pridá `X-Admin-Token` hlavičku.

experiments (harness a scenáre). Samostatný CMake podprojekt mimo produkčného runtime, ktorý linkuje produkčné knižnice ako spotrebiteľ. Skladá sa zo statickej knižnice `experiments_common` a ôsmich samostatných spustiteľných programov (`s1_replay` – `s7_nonce_tamper`). Verejné API tvoria: trieda `ExperimentContext` (`experiment_context.hpp`), ktorá pre každú iteráciu inšanciuje vlastnú kópiu rozhodovacej pipeline (`KeyManager`, `SqliteAccessStore`, `DecisionEngine`, `ProtocolAnomalyDetector`) a vystavuje metódu `processFrame(frameBytes) → DecisionResult` s voliteľným E2E režimom cez `setE2eUrl(url)`; abstraktná trieda `IScenario` v `scenario_runner.hpp` s metódami `setup(ctx)` a `buildTrialFrame(ctx)`, ktoré každý scenár implementuje; a voľná funkcia `runScenario(scenario)` iterujúca cez profily, kroky a behy. Orchestráciu E2E režimu zabezpečuje shell skript `run_e2e.sh`, ktorý pre každý profil generuje dočasný YAML konfig, spúšťa reálny `access_admin` server, seeduje databázu cez REST API a agreguje CSV výstupy.

analysis (Python pipeline pre grafy a tabuľky). Samostatný Python modul mimo CMake buildu, ktorý spracúva CSV výstupy harnessu a generuje publikačné grafy aj LaTeX tabuľky pre prácu. Vstupným bodom je skript `analyze.py` s CLI prepínačmi `-results-dir` (predvolene `build/results`) a `-output-dir` (predvolene `analysis/plots`). Závislosti sú uvedené v `requirements.txt`: `pandas>=2.0` (spracovanie CSV), `matplotlib>=3.7` a `seaborn>=0.13` (grafy), `numpy>=1.24` (numerické výpočty). Globálne konštanty v hornej časti skriptu (`PROFILE_ORDER`, `PROFILE_PALETTE`, `MODE_LABELS`, `ALGO_LABELS`, `SCENARIO_TITLES`) garantujú konzistentné farby, poradie a popisy profilov naprieč všetkými výstupmi.

Pipeline produkuje dve kategórie výstupov do `analysis/plots/`: (a) sedem PNG grafov — `attack_success_heatmap`, `attack_success_by_n`, `quarantine_speed`, `latency_boxplot`, `throughput`, `detector_overhead` a `scenario_summary_table`, všetky uložené pri 300 DPI pre tlač; (b) tri LaTeX tabuľky — `scenario_summary.tex`, `throughput_table.tex`

a `baseline_correctness.tex`, ktoré sa vkladajú priamo do textu diplomovej práce cez `\input{plots/...}`. Tento mechanizmus zabezpečuje, že numerické hodnoty v tabuľkách sú vždy generované z najnovších experimentálnych dát a nemôžu sa „rozísť“ s grafmi.

Vnorenie volaní pri spracovaní karty. Nasledujúci diagram ukazuje postupnosť volaní podprogramov od priloženia karty po zápis do auditu:

```
hw_layer.loop() [ESP32]
  +- rfid.PICC_IsNewCardPresent()
  +- rfid.PICC_ReadCardSerial()
  +- postUidToServer(uid, allow)
    +- connectWiFi() (ak treba)
    +- prefs.putULong64("hw_seq", ...)
    +- hmacSha256Hex(key, "POST /api/hw/uid\n"+body)
    +- http.POST(body) -----> [Wi-Fi / HTTP]
```

```
frontend (port 8080)
  POST /api/hw/uid (lambda v routes_api.cpp)
    +- verifyHwSignature(sig, msg, key)
      +- verifyHmac(msg, key, sig) (privátny helper)
    +- processHwUid(app, req)
      +- buildEncryptedFrameBytes(km, gen, ...)
        +- nonceGenerator.generate(context, seq)
        +- aead.sealWithNonce(payload, aad, nonce)
      +- POST /api/decision/frame --> [HTTP loopback]
```

```
DecisionService (port 8081, loopback 127.0.0.1)
  POST /api/decision/frame (lambda v main.cpp)
    +- DecisionEngine::handleFrameBytes(frameBytes,
```



```
                                replayByReader)

+- FrameHandler::handle(frameBytes)
|   +- parseFrame(bytes, maxCtLen)
|   +- validateReader / validateKeyVersion
|   +- ReplayWindow::contains(seq)          (read-only)
|   +- KeyManager::deriveAeadKey(reader, kv)
|   +- FrameDecryptor::decrypt(frame)
|   |   +- aead.openWithNonce(ct, tag,
|   |   |   aad, nonce)
|   +- ReplayWindow::remember(seq)          (po decrypt)
+- ProtocolAnomalyDetector::isQuarantined
+- ProtocolAnomalyDetector::reportSeq / reportSuccess
+- CardIdHasher::hmacHex(uid)
+- IAccessStore::roleForCardHmac(cardHmacHex)
+- IAccessStore::isAllowed(doorId, role)
+- IAuditLog::append(event)
|   +- getLastEntryHash()
|   +- computeEntryHash(prev, event)
|   +- INSERT audit_log + UPDATE audit_anchor
+- EventBus::push(event)
```

Výstup z DecisionService (DecisionResult) prejde naspäť do frontendu, ktorý odošle {"allow":true|false} firmvéru. Firmvér zavolá signalAllow() alebo signalDeny() a vráti sa do loop().

Vzťah modulov navzájom. Smer závislostí v produkčnom runtime je striktné jednosmerný: access_admin a access_decision sú vyššie vrstvy a používajú nižšie (access_core, access_storage, key_manager, protocol_lib, crypto_lib). Žiadny nižší modul nezávisí od vyššieho, čím je zaručená testovateľnosť každého modulu izolovane (každý má vlastný adresár tests/). config_loader

a `runtime_events` sú „laterálne“ pomocné moduly bez závislostí na rozhodovacej ceste. Modul `experiments` je voči produkčnému runtime paralelný — linkuje produkčné knižnice ako spotrebiteľ, ale nie je nimi linkovaný; vďaka tomu ich môže používať bez rizika, že sa experimentálny kód dostane do produkčného binárneho výstupu. Modul `analysis` je úplne oddelený od C++ ekosystému (Python) a s ostatnými časťami komunikuje len cez CSV súbory na disku.

4 Zoznam použitej literatúry